

PROJECT REVIEW 1

Machine Learning–Based Adaptive Disk I/O Prefetching Using Workload Pattern Classification

Predicting what data a system will need next by first learning how it behaves.

Presented by: Ashvik Mishra 24BAI0342, Shourya Saraf 24BDS0391

What We'll Cover

03

Problem Statement

The gap we are addressing

04-06

Literature review

Key papers on ML-driven storage prefetching

07

Review gap

What existing approaches still miss

08

Objectives

What the project sets out to achieve

09

Novelty & Approach

Why our approach is different

10-11

Architecture & Evaluation Plan

How we'll build and measure it

Why This Problem Matters

Machine Learning–Based Adaptive Disk I/O Prefetching Using Workload Pattern Classification

- Modern applications generate I/O access streams that mix sequential, random, strided, and bursty patterns — often interleaved from multiple processes at once.
- Traditional prefetchers (stride, Markov) are lightweight but cannot model this complexity, leading to poor cache hit rates.
- Deep sequence models (e.g., LSTM) are accurate but computationally heavy and trained offline, making them slow to react when workload behaviour changes.
- There is no lightweight system that first recognises what type of access pattern is occurring and then dynamically adapts its prefetching strategy to it — in real time.

Two Foundational Papers

1

Learning I/O Access Patterns to Improve Prefetching in SSDs

Chakrabortii & Litz, ECML 2020

- Uses an LSTM sequence-to-sequence model to predict future logical block address (LBA) deltas and I/O size from raw block-level traces.
- Reduces the huge LBA space by predicting frequency-ranked “LBA deltas” instead of raw addresses.
- Outperforms stride prefetchers by up to 800× and Markov-chain prefetchers by up to 8× on real data-center traces.
- Model is trained offline in batch mode — not designed to adapt as workload behaviour drifts at runtime.

2

Adaptive Data Prefetching for File Storage Systems Using Online ML

Cyprus Univ. of Technology, 2026

- Proposes an online / streaming ML (SML) framework that extracts features and updates the prefetch model continuously as new I/O arrives.
- Designs streaming algorithms that decide when, how, and what to prefetch, with minimal runtime overhead.
- Tested on production traces from Huawei and Google (SNIA IOTTA); achieves 97% prefetch byte precision and up to 2.8× lower access latency.
- Adapts well to drifting workloads, but treats all access patterns with one general model rather than explicitly recognising workload type.

Two Recent Systems (2024–2026)

3

Baleen: ML Admission & Prefetching for Flash Caches

Wong et al., USENIX FAST 2024

- Trains supervised ML models offline on production traces to decide both flash-cache admission and which blocks to prefetch to SSD.
- Optimises end-to-end disk-head time instead of raw hit rate, splitting caching into admission, prefetching and eviction subproblems.
- On Meta production traces it cuts peak HDD disk-head time by 12% and total cost by 17% against state-of-the-art cache policies.
- Trained offline for datacenter flash caches — it never labels the live access pattern, so it cannot react to workload drift.

4

CAPSULE: Clustering-Assisted Prefetching for Cloud Storage

Ramadhan et al., ACM SIGMETRICS 2026

- Clusters logical block addresses by spatial and temporal proximity, then prefetches neighbouring clusters instead of fixed strides.
- Unsupervised — keeps working where temporal locality is weak and long stack distances defeat recency-based prefetchers.
- On 729 real-world traces it lifts cache hit rate by up to 6.1× and completion time by up to 1.89×, beating rule-based and learned prefetchers.
- Clusters are built offline and prefetching is always aggressive — there is no per-request adaptation when the pattern changes.

Wider Related Work

PAPER	LEARNING	PREDICTION TARGET	MAIN LIMITATION
Soundararajan et al. <i>USENIX ATC 2008</i>	Heuristic · offline	Next block from file / block context	Fixed context patterns with no learning — little adaptivity when behaviour changes.
Akgun et al. <i>arXiv 2021</i>	ML · offline	Read-ahead size	Batch-trained, not streaming; decides only one coarse knob.
Chakrabortii & Litz <i>NVMW 2022</i>	Deep learning · offline	Next page for flash prefetch	Heavy compute, flash-specific, and retrained offline.
Lucas Filho et al. <i>AI4Sys 2023</i>	ML · online	Next block	Early workshop paper — limited detail and evaluation scope.
CrossPrefetch <i>ASPLOS 2024</i>	None · OS mechanism	Prefetch coordination across layers	No learned component; targets concurrency and overhead, not pattern accuracy.

MOTIVATION

The Gap Between Accurate and Adaptive

APPROACH

STRENGTH

LIMITATION

Stride / Markov prefetchers

Fast & simple

Fail badly on complex, interleaved real-world workloads

LSTM sequence model (Paper 1)

Very high accuracy

Trained offline in batch — costly to retrain, slow to adapt

Online streaming ML (Paper 2)

Adapts to drift in real time

Uses one general model — does not classify “what kind” of access pattern is occurring

Gap: no existing system first classifies the workload's access pattern in real time and then adapts its prefetch strategy to that specific pattern.

What We Set Out to Build

1

Classify access patterns

Detect sequential, random, strided, and mixed I/O patterns in real time from streaming trace features.

2

Adapt the prefetch policy

Select and tune a prefetching strategy dynamically based on the classified workload pattern.

3

Learn online

Continuously update the classifier as workload behaviour drifts, avoiding costly offline retraining.

4

Benchmark rigorously

Compare against stride, Markov, and LSTM baselines on accuracy, cache hit ratio, and latency.

Classify First, Then Prefetch

Explicit pattern-classification layer

Neither reference work explicitly labels the workload type; Paper 1 predicts raw LBA sequences and Paper 2 applies one general streaming model to everything. We insert a classification step — sequential / random / strided / mixed — before prefetching decisions are made.

Policy switches per workload class

Instead of one fixed strategy, the prefetch policy (e.g., aggressive read-ahead vs. conservative caching) is selected per detected pattern, combining the strengths of stride, Markov, and learned predictors where each works best.

Lightweight + online

Uses compact, fast-to-update classifiers rather than a full LSTM, keeping the accuracy benefits of pattern-awareness while retaining the real-time adaptability of streaming ML.

System Architecture



Workload classes detected: Sequential · Random · Strided · Mixed / Bursty

- Trace stream carries timestamp, LBA offset, and I/O size for each read/write.
- Lightweight features (deltas, burst rate, locality) are computed on the fly, without heavy preprocessing.
- A fast classifier tags the current access window with a workload pattern label.
- The prefetch policy is chosen per label, then continuously tuned as new data arrives.

How We'll Measure Success

Datasets & Baselines

- MSR Cambridge & SNIA IOTTA block-level I/O traces
- Baselines: Stride prefetcher, Markov-chain prefetcher, offline LSTM model
- Metrics: classification accuracy, prefetch precision/recall, cache hit ratio, access latency

Expected Outcome

- Higher cache hit ratio than heuristic baselines on mixed real-world traces
- Comparable accuracy to LSTM-based prefetching at a fraction of the compute cost
- Faster adaptation to workload drift than a single-model streaming approach

References: Chakrabortii & Litz, "Learning I/O Access Patterns to Improve Prefetching in SSDs," ECML 2020 · "Adaptive Data Prefetching for File Storage Systems Using Online Machine Learning," 2026 · Wong et al., "Baleen: ML Admission & Prefetching for Flash Caches," FAST 2024 · Ramadhan et al., "CAPSULE: Clustering-Assisted Prefetching," SIGMETRICS 2026